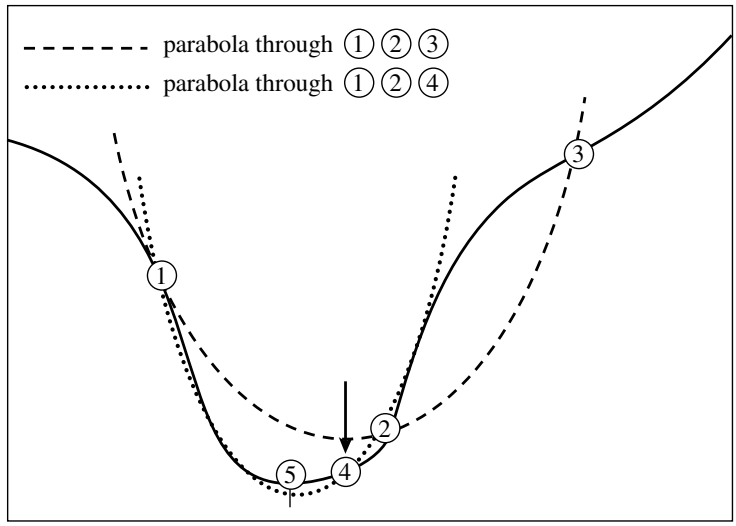

Figure 10.3.1. Convergence to a minimum by inverse parabolic interpolation. A parabola (dashed line) is drawn through the three original points 1,2,3 on the given function (solid line). The function is evaluated at the parabola's minimum, 4, which replaces point 3. A new parabola (dotted line) is drawn through points 1,4,2. The minimum of this parabola is at 5, which is close to the minimum of the function.

You can read the code below to understand the method's logical organization. Mention of a few general principles here may, however, be helpful: Parabolic interpolation is attempted, fitting through the points x, v, and w. To be acceptable, the parabolic step must (i) fall within the bounding interval (a, b), and (ii) imply a movement from the best current value x that is *less* than half the movement of the *step before last*. This second criterion ensures that the parabolic steps are actually converging to something, rather than, say, bouncing around in some nonconvergent limit cycle. In the worst possible case, where the parabolic steps are acceptable but useless, the method will approximately alternate between parabolic steps and golden sections, converging in due course by virtue of the latter. The reason for comparing to the step *before* last seems essentially heuristic: Experience shows that it is better not to "punish" the algorithm for a single bad step if it can make it up on the next one.

Another principle exemplified in the code is never to evaluate the function less than a distance `tol` from a point already evaluated (or from a known bracketing point). The reason is that, as we saw in equation (10.2.2), there is simply no information content in doing so: The function will differ from the value already evaluated only by an amount of order the roundoff error. Therefore, in the code below you will find several tests and modifications of a potential new point, imposing this restriction. This restriction also interacts subtly with the test for "doneness," which the method takes into account.

A typical ending configuration for Brent's method is that a and b are $2 \times x \times$ `tol` apart, with x (the best abscissa) at the midpoint of a and b, and therefore fractionally accurate to $\pm$`tol`.

The calling sequence for `Brent` is exactly analogous to that of `Golden` in the previous section. Indulge us a final reminder that `tol` should generally be no smaller than the square root of your machine's floating-point precision.

mins.h

```
struct Brent : Bracketmethod {
```

Brent's method to find a minimum.

```
    Doub xmin,fmin;
    const Doub tol;
    Brent(const Doub toll=3.0e-8) : tol(toll) {}
    template <class T>
    Doub minimize(T &func)
```

Given a function or functor `f`, and given a bracketing triplet of abscissas `ax`, `bx`, `cx` (such that `bx` is between `ax` and `cx`, and `f(bx)` is less than both `f(ax)` and `f(cx)`), this routine isolates the minimum to a fractional precision of about `tol` using Brent's method. The abscissa of the minimum is returned as `xmin`, and the function value at the minimum is returned as `min`, the returned function value.

```
    {
        const Int ITMAX=100;
        const Doub CGOLD=0.3819660;
        const Doub ZEPS=numeric_limits<Doub>::epsilon()*1.0e-3;
```

Here `ITMAX` is the maximum allowed number of iterations; `CGOLD` is the golden ratio; and `ZEPS` is a small number that protects against trying to achieve fractional accuracy for a minimum that happens to be exactly zero.

```
        Doub a,b,d=0.0,etemp,fu,fv,fw,fx;
        Doub p,q,r,tol1,tol2,u,v,w,x,xm;
        Doub e=0.0;                                  This will be the distance moved on
                                                         the step before last.
        a=(ax < cx ? ax : cx);                       a and b must be in ascending order,
        b=(ax > cx ? ax : cx);                           but input abscissas need not be.
        x=w=v=bx;                                    Initializations...
        fw=fv=fx=func(x);
        for (Int iter=0;iter<ITMAX;iter++) {         Main program loop.
            xm=0.5*(a+b);
            tol2=2.0*(tol1=tol*abs(x)+ZEPS);
            if (abs(x-xm) <= (tol2-0.5*(b-a))) {     Test for done here.
                fmin=fx;
                return xmin=x;
            }
            if (abs(e) > tol1) {                     Construct a trial parabolic fit.
                r=(x-w)*(fx-fv);
                q=(x-v)*(fx-fw);
                p=(x-v)*q-(x-w)*r;
                q=2.0*(q-r);
                if (q > 0.0) p = -p;
                q=abs(q);
                etemp=e;
                e=d;
                if (abs(p) >= abs(0.5*q*etemp) || p <= q*(a-x)
                        || p >= q*(b-x))
                    d=CGOLD*(e=(x >= xm ? a-x : b-x));
```

The above conditions determine the acceptability of the parabolic fit. Here we take the golden section step into the larger of the two segments.

```
                else {
                    d=p/q;                           Take the parabolic step.
                    u=x+d;
                    if (u-a < tol2 || b-u < tol2)
                        d=SIGN(tol1,xm-x);
                }
            } else {
                d=CGOLD*(e=(x >= xm ? a-x : b-x));
            }
            u=(abs(d) >= tol1 ? x+d : x+SIGN(tol1,d));
            fu=func(u);
```

This is the one function evaluation per iteration.

```
            if (fu <= fx) {                          Now decide what to do with our func-
                if (u >= x) a=x; else b=x;               tion evaluation.
                shft3(v,w,x,u);                      Housekeeping follows:
                shft3(fv,fw,fx,fu);
```

```
            } else {
                if (u < x) a=u; else b=u;
                if (fu <= fw || w == x) {
                    v=w;
                    w=u;
                    fv=fw;
                    fw=fu;
                } else if (fu <= fv || v == x || v == w) {
                    v=u;
                    fv=fu;
                }
            }
        }
        throw("Too many iterations in brent");
    }
};
```

Done with housekeeping. Back for another iteration.

CITED REFERENCES AND FURTHER READING:

Brent, R.P. 1973, *Algorithms for Minimization without Derivatives* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2002 (New York: Dover), Chapter 5.[1]

Forsythe, G.E., Malcolm, M.A., and Moler, C.B. 1977, *Computer Methods for Mathematical Computations* (Englewood Cliffs, NJ: Prentice-Hall), §8.2.

10.4 One-Dimensional Search with First Derivatives

Here we want to accomplish precisely the same goal as in the previous section, namely to isolate a functional minimum that is bracketed by the triplet of abscissas (a, b, c), but utilizing an additional capability to compute the function's first derivative as well as its value.

In principle, we might simply search for a zero of the derivative, ignoring the function value information, using a root finder like `rtflsp` or `zbrent` (§9.2 – §9.3). It doesn't take long to reject *that* idea: How do we distinguish maxima from minima? Where do we go from initial conditions where the derivatives on one or both of the outer bracketing points indicate that "downhill" is in the direction *out* of the bracketed interval?

We don't want to give up our strategy of maintaining a rigorous bracket on the minimum at all times. The only way to keep such a bracket is to update it using function (not derivative) information, with the central point in the bracketing triplet always that with the lowest function value. Therefore, the role of the derivatives can only be to help us choose new trial points within the bracket.

One school of thought is to "use everything you've got": Compute a polynomial of relatively high order (cubic or above) that agrees with some number of previous function and derivative evaluations. For example, there is a unique cubic that agrees with function and derivative at two points, and one can jump to the interpolated minimum of that cubic (if there is a minimum within the bracket). Suggested by Davidon and others, formulas for this tactic are given in [1].

We like to be more conservative than this. Once superlinear convergence sets in, it hardly matters whether its order is moderately lower or higher. In practical

problems that we have met, most function evaluations are spent in getting globally close enough to the minimum for superlinear convergence to commence. So we are more worried about all the funny "stiff" things that high-order polynomials can do (cf. Figure 3.0.1(b)), and about their sensitivities to roundoff error.

This leads us to use derivative information only as follows: The sign of the derivative at the central point of the bracketing triplet (a, b, c) indicates uniquely whether the next test point should be taken in the interval (a, b) or in the interval (b, c). The value of this derivative and of the derivative at the second-best-so-far point are extrapolated to zero by the secant method (inverse linear interpolation), which by itself is superlinear of order 1.618. (The golden mean again: See [1], p. 57.) We impose the same sort of restrictions on this new trial point as in Brent's method. If the trial point must be rejected, we *bisect* the interval under scrutiny.

Yes, we are fuddy-duddies when it comes to making flamboyant use of derivative information in one-dimensional minimization. But we have met too many functions whose computed "derivatives" *don't* integrate up to the function value and *don't* accurately point the way to the minimum, usually because of roundoff errors, sometimes because of truncation error in the method of derivative evaluation.

You will see that the following routine is closely modeled on `Brent` in the previous section. One difference is in the input to the routine. Whereas `Brent` takes either a function or functor argument, `Dbrent` takes only a functor. The functor returns not only the function value, by overloading `operator()`, but also the derivative as the member function `df`. For example, here's how you would code the function x^2:

```
struct Funcd {                          Name Funcd is arbitrary.
    Doub operator() (const Doub x) {
        return x*x;
    }
    Doub df(const Doub x) {
        return 2.0*x;
    }
};
```

To invoke `Dbrent`, you need statements like the following:

```
Dbrent dbrent;
Funcd f;
dbrent.bracket(a,b,f);
xmin=dbrent.minimize(f);
```

The value of the function at the minimum is available in `dbrent.fmin` as usual. Here is the routine:

mins.h

```
struct Dbrent : Bracketmethod {
```

Brent's method to find a minimum, modified to use derivatives.

```
    Doub xmin,fmin;
    const Doub tol;
    Dbrent(const Doub toll=3.0e-8) : tol(toll) {}
    template <class T>
    Doub minimize(T &funcd)
```

Given a functor `funcd` that computes a function and also its derivative function `df`, and given a bracketing triplet of abscissas `ax`, `bx`, `cx` [such that `bx` is between `ax` and `cx`, and `f(bx)` is less than both `f(ax)` and `f(cx)`], this routine isolates the minimum to a fractional precision of about `tol` using a modification of Brent's method that uses derivatives. The abscissa of the minimum is returned as `xmin`, and the minimum function value is returned as `min`, the returned function value.

```
    {
        const Int ITMAX=100;
```

```
const Doub ZEPS=numeric_limits<Doub>::epsilon()*1.0e-3;
Bool ok1,ok2;                                   Will be used as flags for whether pro-
Doub a,b,d=0.0,d1,d2,du,dv,dw,dx,e=0.0;         posed steps are acceptable or not.
Doub fu,fv,fw,fx,olde,tol1,tol2,u,u1,u2,v,w,x,xm;
```

Comments following will point out only differences from the routine in `Brent`. Read that routine first.

```
a=(ax < cx ? ax : cx);
b=(ax > cx ? ax : cx);
x=w=v=bx;
fw=fv=fx=funcd(x);
dw=dv=dx=funcd.df(x);                           All our housekeeping chores are dou-
for (Int iter=0;iter<ITMAX;iter++) {                bled by the necessity of moving
    xm=0.5*(a+b);                                   aorund derivative values as well
    tol1=tol*abs(x)+ZEPS;                           as function values.
    tol2=2.0*tol1;
    if (abs(x-xm) <= (tol2-0.5*(b-a))) {
        fmin=fx;
        return xmin=x;
    }
    if (abs(e) > tol1) {
        d1=2.0*(b-a);                           Initialize these d's to an out-of-bracket
        d2=d1;                                      value.
        if (dw != dx) d1=(w-x)*dx/(dx-dw);      Secant method with one point.
        if (dv != dx) d2=(v-x)*dx/(dx-dv);      And the other.
        Which of these two estimates of d shall we take? We will insist that they be
        within the bracket, and on the side pointed to by the derivative at x:
        u1=x+d1;
        u2=x+d2;
        ok1 = (a-u1)*(u1-b) > 0.0 && dx*d1 <= 0.0;
        ok2 = (a-u2)*(u2-b) > 0.0 && dx*d2 <= 0.0;
        olde=e;                                 Movement on the step before last.
        e=d;
        if (ok1 || ok2) {                       Take only an acceptable d, and if
            if (ok1 && ok2)                         both are acceptable, then take
                d=(abs(d1) < abs(d2) ? d1 : d2); the smallest one.
            else if (ok1)
                d=d1;
            else
                d=d2;
            if (abs(d) <= abs(0.5*olde)) {
                u=x+d;
                if (u-a < tol2 || b-u < tol2)
                    d=SIGN(tol1,xm-x);
            } else {                            Bisect, not golden section.
                d=0.5*(e=(dx >= 0.0 ? a-x : b-x));
                Decide which segment by the sign of the derivative.
            }
        } else {
            d=0.5*(e=(dx >= 0.0 ? a-x : b-x));
        }
    } else {
        d=0.5*(e=(dx >= 0.0 ? a-x : b-x));
    }
    if (abs(d) >= tol1) {
        u=x+d;
        fu=funcd(u);
    } else {
        u=x+SIGN(tol1,d);
        fu=funcd(u);
        if (fu > fx) {                          If the minimum step in the downhill
            fmin=fx;                                direction takes us uphill, then
            return xmin=x;                          we are done.
        }
```